QueryGenie: A Natural-Language-to-SQL Interface that Knows When It Does Not Know

N V V S S Jayakanth Kamisetti, Boddupally Narendar, M Praneeth Reddy
Department of Computer Science & Information Technology
Koneru Lakshmaiah Education Foundation (KL University), Hyderabad, India
Team 18 · IDs 2320090050, 2320090009, 2320090018

Supervisor: M Parameswar
Associate Professor, Department of CSIT
Koneru Lakshmaiah Education Foundation (KL University), Hyderabad, India

Abstract—Open-weight text-to-SQL models return one query per question with no sign of whether it can be trusted. We reproduce the open CodeS-1B model, then make it say when it is unsure. On the 1,034 Spider development questions we measure 77.8% execution accuracy against the paper's 77.9%, using the authors' own evaluator. We execute the four candidate queries that beam search already produces, count how many return the same result, and abstain when that agreement is low. At 90.4% coverage the system is right 83.3% of the time instead of 77.8%, and gives 156 wrong answers instead of 230; over ten calibration/test splits the gain over random abstention is +5.46 points (paired t-test $p = 7.7 \times 10^{-10}$, Wilcoxon $p = 9.8 \times 10^{-4}$, bootstrap 95% CI $[+4.5, +6.7]$). Because an examiner noted that candidates are often copies, we measure how many are different (all four in 56% of questions) and repeat the study over different queries only, with CodeS-3B, and with eight sampled candidates; the gain holds in each. The pipeline reaches 69.5% on Spider-Realistic (paper: 70.9%) and runs on a live PostgreSQL database through a read-only SQL translation, where 94.4% of the Spider gold queries return the same rows as on SQLite. It ships as a desktop application that runs locally. Test-suite accuracy and a full BIRD evaluation are TBD.

Keywords—text-to-SQL, selective prediction, abstention, execution agreement, reproducibility, CodeS, Spider

I. Introduction

A natural-language interface to a database is only useful if its users can tell when to believe it. Text-to-SQL models are usually compared by how often they are right when they answer every question [1], [2], but a wrong query is more costly than no query: it returns a plausible table that looks like an answer. The strongest systems rely on closed hosted models [3]–[5], which means a company's schema is sent to a third party. CodeS [6] narrows that gap with open models of 1B to 15B parameters, but like the others it returns one query with uniform apparent confidence.

This paper does three things.

1) Reproduction. We reproduce SFT CodeS-1B on Spider [7] within 0.1 points of the published figure, after auditing our harness line by line against the authors' code and fixing the errors that audit found (Section III).
2) Confidence-aware abstention. We use the candidates CodeS already generates: the system executes all of them, measures how many agree on a result, and declines to answer, with a clarifying choice, when agreement is low. We evaluate this as an accuracy–coverage trade-off with a pre-registered significance protocol (Section IV).
3) Candidate diversity and other settings. We test the claim behind the method — that agreement among candidates carries information — when the candidates are partly duplicates, with a larger model, with sampled candidates, on rewritten questions and on a live PostgreSQL server (Sections V–VI).

All numbers below come from logged runs (seed 42, hardware and runtime recorded for each) and the scripts that analyse them; a number we did not measure is marked TBD.

II. Related Work

Spider [7] is the standard cross-domain benchmark, scored today by execution rather than by string match, with test-suite accuracy [8] as a stricter variant. Spider-Realistic [9], Spider-Syn [10] and BIRD [11] probe robustness to wording, synonyms and large real-world databases. Methods moved from graph encoders [12], [13] and constrained decoding [14] to large language models, prompted [3], [4], [15], [16] or fine-tuned [6], [17]. BRIDGE [18]-style value matching, which CodeS keeps, adds matched cell values to the prompt.

Our abstention rule combines two older ideas: self-consistency, which trusts the answer that most sampled reasoning paths reach [19], and selective classification, which lets a model decline inputs and reports the accuracy–coverage trade-off [20]. Execution-guided repair, as in DIN-SQL [3], addresses queries that fail; we measure it and find it addresses a small share of CodeS's errors (Section IV). To our reading of the surveys [2], [21]–[24], none of the systems reviewed reports an accuracy–coverage curve for the generated query; the claim here is limited to those works.

TABLE I. Execution accuracy (EX) on Spider dev, 1,034 questions.

Run	Setting	EX
T2-001	no value retriever, simple metric	64.8%
T2-003	full prompt, simple metric	73.8%
T2-005	audited harness, official metric	77.8%
	paper, CodeS-1B	77.9%
T2-004	CodeS-3B, official metric	81.8%
	paper, CodeS-3B	83.4%

III. Reproducing CodeS

Pipeline. For each question CodeS builds a prompt from the database: every table with its columns, types, keys and representative values, human-readable column descriptions, and stored values matched to words in the question by the authors' value retriever. The model generates four candidates by beam search and returns the first that SQLite accepts (EXPLAIN QUERY PLAN), otherwise the first beam. We use the authors' fine-tuned seeklhy/codes-1b-spider, fp16, one Tesla T4.

Harness audit. Our first runs scored 64.8–73.8%. We compared the harness line by line with the authors' code and found a bug in our port (numbers were never matched by the value retriever, changing the prompt for 12.4% of questions), missing column descriptions, and a simplified scoring metric that was not the paper's. We fixed these, switched to the official evaluator [8] (execution match), and kept the earlier runs in the log, marked as superseded.

Result. The corrected pipeline reaches 77.8% execution accuracy (804/1034; run T2-005, re-run from scratch as T2-006 with an identical result), against 77.9% in the paper. Table I shows the effect of each fix and of the model size. The 3B model reaches 81.8% (paper: 83.4%, −1.6 points). Test-suite accuracy was not computed (TBD).

Ablation. Removing one component at a time (official metric, same model and seed) lowers accuracy in every case: foreign keys −0.9 points, representative column values −1.4, candidate selection −1.8, value retriever −1.9. These differences are 9–20 questions each and were not tested for significance; the order of the two smallest effects should not be over-read. Accuracy falls with difficulty: 90.3% on easy, 83.6% medium, 64.9% hard and 56.6% extra-hard questions.

Deviations. We match values by scanning all distinct values rather than the authors' Lucene BM25 pre-filter; we do not apply the optional schema classifier; and we use newer PyTorch and transformers versions on one T4 instead of eight A800 GPUs.

IV. Confidence-Aware Abstention

Method. Of the 97.3% of queries that execute, many still return the wrong rows, so repairing failures cannot fix most errors. Execution-guided repair, tried as in DIN-SQL, sent back 377 failed queries over 211 questions; 61 then ran and one more answer became correct (77.76% → 77.85%). We therefore turned to the queries that run but are wrong. All candidates are executed (read-only, on the original database); the agreement a of the returned answer is the share of candidates whose result equals it, and 0 if the returned query fails. The confidence is $c = a + 0.01\,p_b$, where p_b is the softmax beam probability of the returned candidate (a tie-breaker only). The system abstains when $c < \tau$: it then shows no table, but lists the distinct results of the candidates as readings for the user to choose from. No model is retrained (Fig. 1).

TABLE II. Answering everything vs. abstaining (CodeS-1B, run T2-006).

Setting	Answered	Acc. on answered	Wrong
Answer everything	1,034 (100%)	77.76%	230
Balanced ($\tau = 0.2526$)	935 (90.4%)	83.32%	156
Strict ($\geq$3 of 4 agree)	649 (62.8%)	86.13%	90
Very strict (all 4 agree)	412 (39.8%)	88.35%	48

Protocol. Fixed before the results were seen: ten random 50/50 calibration/test splits of the 1,034 questions; τ chosen on the calibration half for at least 90% coverage and applied to the test half; accuracy on answered questions compared with random abstention at the same coverage (the model's accuracy on that half); one-sided paired t-test and Wilcoxon signed-rank test at $\alpha = 0.05$; and a question-level bootstrap (5,000 resamples) that does not rely on the splits being independent. The deployed threshold, $\tau = 0.2526$, is the median over the ten calibration halves. In words: abstain when no candidate runs, or when only one of four supports the answer and its beam probability is below about 0.26.

Results. Table II and Fig. 2 show the outcome. At 90.4% coverage the system answers correctly 83.3% of the time against 77.8% for answering everything; wrong answers fall from 230 to 156 (−32%). Of the 99 questions it declines, 74 would have been wrong and 25 correct. The mean gain over random abstention is +5.46 points (SD 0.71), positive in all ten splits: paired $t = 24.4$, $p = 7.7 \times 10^{-10}$; Wilcoxon $W = 55$, $p = 9.8 \times 10^{-4}$ (the smallest value possible with ten pairs); bootstrap 95% CI $[+4.51, +6.71]$. We reject the null hypothesis of no gain. Beam probability alone gives a smaller gain (+3.08 points); agreement is higher than beam probability by 2.37 points on the same splits (paired $t = 6.31$, $p = 1.4 \times 10^{-4}$).

Limits of the signal. Agreement is not calibrated at the top: 11.7% of answers on which all four candidates agree are still wrong. Over the whole curve, beam probability ranks answers slightly better (area under the risk–coverage curve 0.0905 against 0.1025 for agreement; random 0.2224, perfect ranking 0.0269); agreement is ahead from full coverage down to about 74% coverage, the region where the operating point lies. Agreement takes only five values, so intermediate coverages rely on the beam-probability tie-breaker.

V. Are the Candidates Really Different?

At the review of this work an examiner observed that, for some questions, only two of the four candidates were different queries, and asked why the number is four. This matters, because agreement among copies measures

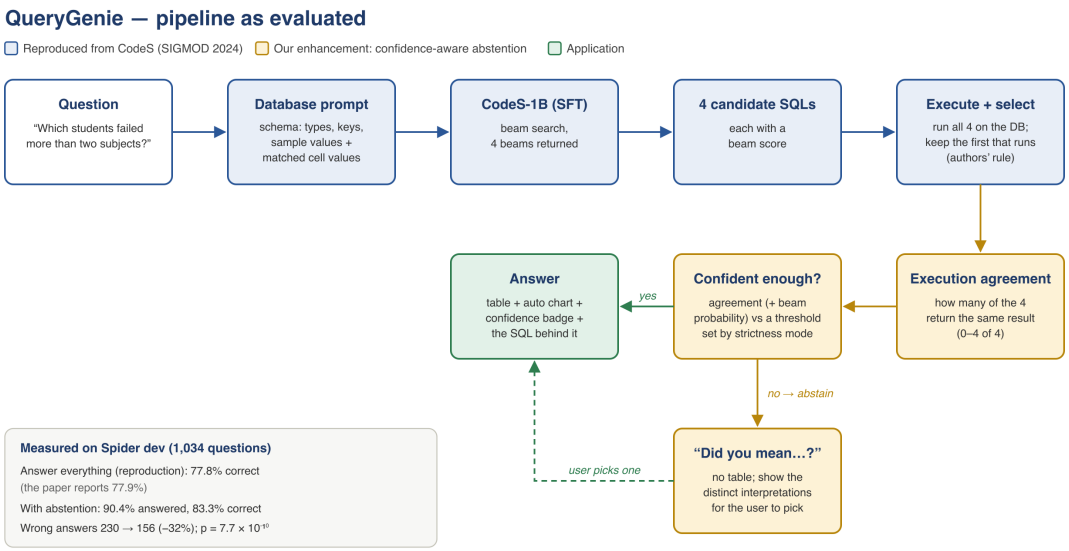


Fig. 1. System architecture: the reproduced CodeS components, the abstention step added here, and the application.

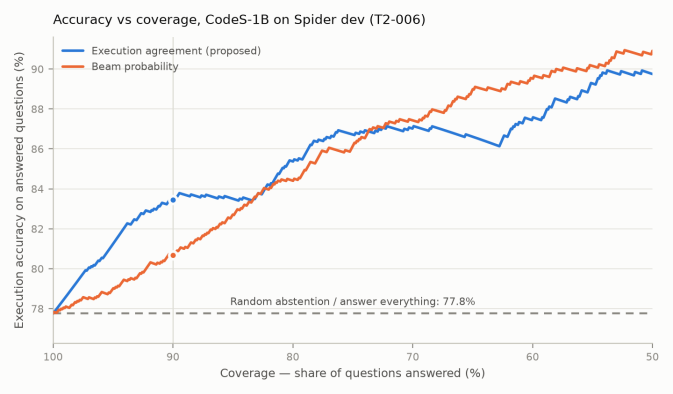


Fig. 2. Accuracy on answered questions against coverage (CodeS-1B, Spider dev).

nothing. We re-executed every saved candidate of run T2-006.

Duplicates. The four beams are four different queries for only 577 of 1,034 questions (55.8%); 42 questions have one distinct query, 190 two and 225 three. For CodeS-3B the counts are 57, 234, 249 and 494.

Agreement over different queries. We recomputed agreement ("v2") counting each different query once and comparing results up to row and column order, which also corrects 68 questions where only the column order hid agreement. Over different queries, accuracy rises with the number that agree: 0 agree: 0.0% (28 questions); 1: 71.7% (244); 2: 78.7% (367); 3: 83.1% (207); 4: 89.4% (188). The abstention gain is unchanged in kind: +5.14 points as a share ($p = 1.7{\times}10^{-9}$) and +5.66 as a count ($p = 4{\times}10^{-11}$), against +5.46 for the evaluated rule. The conclusion does not depend on duplicates.

Another model. For CodeS-3B the same analysis gives agreement +3.18 points ($p = 2.9 \times 10^{-6}$; 89.7% coverage, 85.2% accuracy on answered) and, over different queries, +3.88 ($p = 3.6 \times 10^{-7}$). Abstention helps the larger and the smaller model, by less for the larger one, which is already right more often.

Sampled candidates. We also drew eight candidates by sampling (temperature 0.8) instead of beams (run SM-008). Plain accuracy is 77.08% (beams: 77.76%); a correct query is among the eight for 881 questions (85.2%) against 859 (83.1%) among four beams. Majority vote over the eight reaches 77.66%, whereas majority vote over four beams was worse than the first-executable rule (75.73% against 77.76%). Agreement abstention holds (+5.57 points, $p = 1.7 \times 10^{-7}$, 82.5% accuracy at 90.0% coverage), and the 491 questions on which all eight agree are correct 93.5% of the time.

Why four? Four is what CodeS's inference code returns and what our threshold was calibrated on. The application now lets the user choose 2–12 candidates, over-generating beams (or sampling at rising temperature until the queries differ) so that candidates are different queries wherever the model has more than one way to answer. This was done after the evaluation: the measured accuracy and threshold are for four beams, and other counts are uncalibrated. A run with eight beams printed 78.8% accuracy before it failed while saving its results; it is being re-run, and no further number from it is reported (TBD).

VI. Beyond Spider

Rewritten questions. Spider questions often name the columns. On the 508 development questions that Spider-Realistic [9] rewrites without column names, the pipeline (run on an Apple M4) reaches 69.5% (353/508) against the paper's 70.9% (−1.4 points). On the original wording of the same 508 questions it reaches 72.8% (370/508), so the wording costs 3.3 points (bootstrap 95% CI [0.8, 5.9];

31 questions right only in the original wording, 14 only in the realistic one; exact McNemar $p = 0.016$). Agreement stays informative there: accuracy is 86.9% when all four candidates agree and 47.1% when one does, and the balanced setting lowers wrong answers from 155 to 102. A run on BIRD Mini-Dev [11] was stopped after 18 of 500 questions and is not reported (TBD).

A live database. CodeS writes SQLite. For a server database we translate each query with sqlglot, accept only one read-only statement, and run it in a read-only transaction with a timeout and a row limit; the schema and a value index are read once at connection time. We copied the 20 Spider development databases into PostgreSQL 14 and ran every gold query and every saved prediction on both engines. With the final rewrites (string literals, case-insensitive LIKE, aggregate grouping) 94.4% of gold queries (976/1034) and 93.7% of predictions that run on SQLite (943/1006) return the same rows on PostgreSQL, up from 67.0% and 71.1% for plain translation; most remaining mismatches are ORDER BY…LIMIT queries with tied rows. End-to-end accuracy and calibration on PostgreSQL, and MySQL, are not measured (TBD); a qualitative run of twelve questions on one database matched the gold answer ten times.

The application. The system ships as a desktop application (macOS; a Windows build is configured but untested) that runs the same evaluated code, shows how many of the queries agree, executes and displays every candidate, offers the readings when it abstains, and runs fully on the local machine with open weights, so the schema is not sent to a third party. Its models, thresholds and candidate count are user settings; only CodeS-1B with four beams has measured accuracy.

VII. Limitations

(1) One benchmark and, for the main result, one model; the threshold was calibrated on the same development set it is reported on, which the split protocol and bootstrap address but cannot remove. (2) The ten splits reuse the same 1,034 questions. (3) Agreement can be unanimous and wrong (11.7%), and is a coarser ranking than beam probability across the whole curve. (4) The ablation differences are small and untested for significance. (5) Test-suite accuracy, BIRD, MySQL and end-to-end PostgreSQL accuracy are TBD, and the larger CodeS models were not run. (6) Candidate counts other than four, and models other than CodeS-1B, are uncalibrated.

VIII. Conclusion and Future Work

An open 1B-parameter text-to-SQL model can be reproduced to within 0.1 points of its published accuracy and made to decline the questions it is likely to get wrong: at 90.4% coverage it is right 83.3% of the time instead of 77.8%, with a third fewer wrong answers, using only the candidates it already generates and no retraining. The result survives duplicated candidates, a larger model and sampled candidates, and the system works on rewritten questions and a live server database. Next steps are the results marked TBD, calibrating the threshold for other candidate counts, per-database adaptation, and role-based access when connected to a company database.

Acknowledgment

The authors thank M Parameswar for supervision, and the Department of Computer Science & Information Technology, KL University, for the project framework.

References

[1] N. Deng, Y. Chen, and Y. Zhang, "Recent Advances in Text-to-SQL: A Survey of What We Have and What We Expect," arXiv preprint arXiv:2208.10099, 2022, cOLING 2022. [Online]. Available: https://arxiv.org/abs/2208.10099

[2] Z. Hong, Z. Yuan, Q. Zhang, H. Chen, J. Dong, F. Huang, and X. Huang, "Next-Generation Database Interfaces: A Survey of LLM-based Text-to-SQL," arXiv preprint arXiv:2406.08426, 2024, tKDE 2025. [Online]. Available: https://arxiv.org/abs/2406.08426

[3] M. Pourreza and D. Rafiei, "DIN-SQL: Decomposed In-Context Learning of Text-to-SQL with Self-Correction," arXiv preprint arXiv:2304.11015, 2023, neurIPS 2023. [Online]. Available: https://arxiv.org/abs/2304.11015

[4] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-SQL Empowered by Large Language Models: A Benchmark Evaluation," arXiv preprint arXiv:2308.15363, 2023, vLDB 2024. [Online]. Available: https://arxiv.org/abs/2308.15363

[5] M. Pourreza, H. Li, R. Sun, Y. Chung, S. Talaei, G. T. Kakkar, Y. Gan, A. Saberi, F. Ozcan, and S. O. Arik, "CHASE-SQL: Multi-Path Reasoning and Preference Optimized Candidate Selection in Text-to-SQL," arXiv preprint arXiv:2410.01943, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2410.01943

[6] H. Li, J. Zhang, H. Liu, J. Fan, X. Zhang, J. Zhu, R. Wei, H. Pan, C. Li, and H. Chen, "CodeS: Towards Building Open-source Language Models for Text-to-SQL," arXiv preprint arXiv:2402.16347, 2024, sIGMOD 2024. [Online]. Available: https://arxiv.org/abs/2402.16347

[7] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman, Z. Zhang, and D. Radev, "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task," arXiv preprint arXiv:1809.08887, 2018, eMNLP 2018. [Online]. Available: https://arxiv.org/abs/1809.08887

[8] R. Zhong, T. Yu, and D. Klein, "Semantic Evaluation for Text-to-SQL with Distilled Test Suites," arXiv preprint arXiv:2010.02840, 2020, eMNLP 2020. [Online]. Available: https://arxiv.org/abs/2010.02840

[9] X. Deng, A. H. Awadallah, C. Meek, O. Polozov, H. Sun, and M. Richardson, "Structure-Grounded Pretraining for Text-to-SQL," arXiv preprint arXiv:2010.12773, 2020, nAACL 2021. [Online]. Available: https://arxiv.org/abs/2010.12773

[10] Y. Gan, X. Chen, Q. Huang, M. Purver, J. R. Woodward, J. Xie, and P. Huang, "Towards Robustness of Text-to-SQL Models against Synonym Substitution," arXiv preprint arXiv:2106.01065, 2021, aCL 2021. [Online]. Available: https://arxiv.org/abs/2106.01065

[11] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Cao, R. Geng, N. Huo, X. Zhou et al., "Can LLM Already Serve as A Database Interface? A BIg Bench for Large-Scale Database Grounded Text-to-SQLs," arXiv preprint arXiv:2305.03111, 2023, neurIPS 2023. [Online]. Available: https://arxiv.org/abs/2305.03111

[12] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers," arXiv preprint arXiv:1911.04942, 2019, aCL 2020. [Online]. Available: https://arxiv.org/abs/1911.04942

[13] R. Cao, L. Chen, Z. Chen, Y. Zhao, S. Zhu, and K. Yu, "LGESQL: Line Graph Enhanced Text-to-SQL Model with Mixed Local and Non-Local Relations," arXiv preprint arXiv:2106.01093, 2021, aCL 2021. [Online]. Available: https://arxiv.org/abs/2106.01093

[14] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models," arXiv preprint arXiv:2109.05093, 2021, eMNLP 2021. [Online]. Available: https://arxiv.org/abs/2109.05093

[15] B. Wang, C. Ren, J. Yang, X. Liang, J. Bai, L. Chai, Z. Yan, Q.-W. Zhang, D. Yin, X. Sun, and Z. Li, "MAC-SQL: A Multi-Agent Collaborative Framework for Text-to-SQL," arXiv preprint arXiv:2312.11242, 2023, aCL 2025. [Online]. Available: https://arxiv.org/abs/2312.11242

[16] S. Talaei, M. Pourreza, Y.-C. Chang, A. Mirhoseini, and A. Saberi, "CHESS: Contextual Harnessing for Efficient SQL Synthesis," arXiv preprint arXiv:2405.16755, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2405.16755

[17] H. Li, J. Zhang, C. Li, and H. Chen, "RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL," arXiv preprint arXiv:2302.05965, 2023, aAAI 2023. [Online]. Available: https://arxiv.org/abs/2302.05965

[18] X. V. Lin, R. Socher, and C. Xiong, "Bridging Textual and Tabular Data for Cross-Domain Text-to-SQL Semantic Parsing," arXiv preprint arXiv:2012.12627, 2020, eMNLP 2020. [Online]. Available: https://arxiv.org/abs/2012.12627

[19] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery, and D. Zhou, "Self-Consistency Improves Chain of Thought Reasoning in Language Models," arXiv preprint arXiv:2203.11171, 2022, iCLR 2023. [Online]. Available: https://arxiv.org/abs/2203.11171

[20] Y. Geifman and R. El-Yaniv, "Selective Classification for Deep Neural Networks," arXiv preprint arXiv:1705.08500, 2017, neurIPS 2017. [Online]. Available: https://arxiv.org/abs/1705.08500

[21] X. Liu, S. Shen, B. Li, P. Ma, R. Jiang, Y. Zhang, J. Fan, G. Li, N. Tang, and Y. Luo, "A Survey of Text-to-SQL in the Era of LLMs: Where are we, and where are we going?" arXiv preprint arXiv:2408.05109, 2024, tKDE 2025. [Online]. Available: https://arxiv.org/abs/2408.05109

[22] A. Mohammadjafari, A. S. Maida, and R. Gottumukkala, "From Natural Language to SQL: Review of LLM-based Text-to-SQL Systems," arXiv preprint arXiv:2410.01066, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2410.01066

[23] L. Shi, Z. Tang, N. Zhang, X. Zhang, and Z. Yang, "A Survey on Employing Large Language Models for Text-to-SQL Tasks," arXiv preprint arXiv:2407.15186, 2024, arXiv 2024. [Online]. Available: https://arxiv.org/abs/2407.15186

[24] B. Qin, B. Hui, L. Wang, M. Yang, J. Li, B. Li, R. Geng, R. Cao, J. Sun, L. Si, F. Huang, and Y. Li, "A Survey on Text-to-SQL Parsing: Concepts, Methods, and Future Directions," arXiv preprint arXiv:2208.13629, 2022, tKDE 2022. [Online]. Available: https://arxiv.org/abs/2208.13629